TRƯỜNG ĐẠI HỌC ĐÀ LẠT
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO GIỮA KỲ

MÔN KIỂM THỬ PHẦN MỀM

KIỂM THỬ RACE CONDITION CHO HỆ THỐNG ĐẶT LỊCH CAL.COM

Đề tài R05-K15 · Nhóm 14

Giảng viên: **Nguyễn Thế Lâm**

Sinh viên thực hiện:

2312741 – Nguyễn Thanh Tâm

2312800 – Võ Công Vinh

2312774 – Nguyễn Đức Tín

2312796 – Ngô Quốc Trường Vĩ

Đà Lạt, tháng 10 năm 2026

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

LỜI CẢM ƠN

Trước hết chúng em xin gửi lời cảm ơn đến thầy Nguyễn Thế Lâm, người đã giao đề tài, hướng dẫn và góp ý cho nhóm trong suốt quá trình tìm hiểu hệ thống, thiết kế và thực hiện kiểm thử. Những yêu cầu của thầy về việc phải kiểm thử thật, lỗi tìm được phải là lỗi thật và có bằng chứng kiểm chứng được đã giúp nhóm hiểu rõ hơn cách làm việc của một đội đảm bảo chất lượng.

Do thời gian và kiến thức của nhóm còn hạn chế, báo cáo không thể tránh khỏi thiếu sót. Chúng em kính mong nhận được sự chỉ bảo thêm của thầy cũng như góp ý của các bạn để nhóm hoàn thiện đề tài hơn nữa.

Một lần nữa, nhóm chúng em xin chân thành cảm ơn các thầy cô trong khoa Công nghệ thông tin, Trường Đại học Đà Lạt, đã tận tình giảng dạy và truyền đạt những kiến thức cần thiết trong suốt thời gian chúng em học tập tại trường.

MỤC LỤC

DANH MỤC HÌNH ẢNH 7

DANH MỤC BẢNG BIỂU 8

MỞ ĐẦU 10

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI 11

1.1. Giới thiệu đề tài 11

1.2. Lý do chọn đề tài 11

1.3. Mục tiêu đề tài 12

1.4. Phạm vi nghiên cứu 12

1.5. Cấu trúc báo cáo và đối chiếu với yêu cầu của đề bài 13

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ 14

2.1. Race condition và kiểm thử đồng thời 14

2.2. Hệ thống đặt lịch và các cơ chế bảo vệ 14

2.3. Công nghệ sử dụng 15

2.3.1. Cal.com 15

2.3.2. PostgreSQL 15

2.3.3. k6 15

2.3.4. Node.js và thư viện pg 16

2.3.5. Git và GitHub Projects 16

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ KIỂM THỬ 17

3.1. Phân tích yêu cầu kiểm thử 17

3.1.1. Yêu cầu chức năng 17

3.1.2. Yêu cầu phi chức năng của bộ test 17

3.2. Phân tích hệ thống được kiểm thử 18

3.2.1. Mục tiêu, tác nhân và use case chính 18

3.2.2. Cây thư mục và module 19
3.2.3. Phụ thuộc chính 19
3.2.4. Luồng dữ liệu của nghiệp vụ đặt lịch 20
3.2.5. Kiến trúc và các module liên quan đến kiểm thử 20
3.3. Thiết kế kiểm thử 22
3.3.1. Nguyên lý và lý do chọn phạm vi 22
3.3.2. Giả định 23
3.3.3. Rủi ro và cách xử lý 23
3.3.4. Quy trình test 23
3.3.5. Invariant và tiêu chí pass/fail 24
3.3.6. Danh sách và đặc tả test case 25
3.3.7. Cơ sở dữ liệu liên quan 28
3.4. Kế hoạch kiểm thử và tính khả thi 29
3.4.1. Kế hoạch và lịch thực hiện 29
3.4.2. Tính khả thi và đề nghị duyệt 29
CHƯƠNG 4. XÂY DỰNG VÀ THỰC HIỆN KIỂM THỬ 31
4.1. Cấu trúc thư mục và tổ chức mã nguồn 31
4.2. Môi trường, triển khai và bằng chứng hệ thống chạy 31
4.3. Xây dựng bộ test 32
4.3.1. Dữ liệu test 32
4.3.2. Gửi yêu cầu đồng thời bằng k6 33
4.3.3. Kiểm tra invariant và nhóm đối chứng 33
4.3.4. Báo cáo và lệnh chạy 33
4.4. Thực hiện kiểm thử và kết quả 34
4.4.1. Kết quả chạy đồng thời và tuần tự 34

4.4.2. Chạy lại trên máy Windows 35
4.4.3. Độ nhạy và cửa sổ race 37
4.4.4. Chi phí kiểm thử 37
4.5. Phân tích lỗi 38
4.5.1. Danh sách lỗi 38
4.5.2. Tái hiện độc lập bằng curl 39
4.5.3. Nguyên nhân gốc 39
4.5.4. Thí nghiệm khắc phục 40
4.6. Quản lý dự án, nhiệm vụ và đóng góp của từng thành viên 41
4.7. Khả năng tái lập 42
KẾT LUẬN 44
1. Kết quả đạt được 44
2. Hạn chế và hướng phát triển 44
TÀI LIỆU THAM KHẢO 46
PHỤ LỤC 47

DANH MỤC HÌNH ẢNH

Hình 1. Sơ đồ use case chính của Cal.com 18
Hình 2. Sơ đồ container của Cal.com 21
Hình 3. Hệ thống được kiểm thử và vị trí của bộ test 21
Hình 4. Khe hở kiểm tra rồi mới ghi khi hai khách đặt hai giờ chồng nhau 22
Hình 5. Quy trình test gồm bảy bước 24
Hình 6. Cấu trúc thư mục bộ test 31
Hình 7. Số vòng vi phạm của từng test case trên hai máy, hai chế độ 35
Hình 8. Kết quả chạy đồng thời trên máy Windows 36
Hình 9. Kết quả chạy tuần tự trên máy Windows 36
Hình 10. Tỉ lệ vòng vi phạm theo độ trễ của yêu cầu thứ hai 37
Hình 11. Bảng công việc GitHub Projects của nhóm 42

DANH MỤC BẢNG BIỂU

Bảng 1. Phạm vi kiểm thử của đề tài 13
Bảng 2. Đối chiếu các phần A đến H của đề bài với các mục của báo cáo 13
Bảng 3. Các use case chính của Cal.com 18
Bảng 4. Các thư mục chính của Cal.com 19
Bảng 5. Các phụ thuộc chính của Cal.com v6.2.0 19
Bảng 6. Rủi ro của kế hoạch kiểm thử và cách xử lý 23
Bảng 7. Các bước của quy trình test 24
Bảng 8. Các invariant dùng làm tiêu chí đúng sai 25
Bảng 9. Các loại cuộc hẹn dùng cho kiểm thử 25
Bảng 10. Danh sách test case 26
Bảng 11. Bảng đặc tả TC-01: nhiều người đặt cùng một giờ 26
Bảng 12. Bảng đặc tả TC-02: các giờ chồng lấn 26
Bảng 13. Bảng đặc tả TC-03: tranh ghế đầu tiên 27
Bảng 14. Bảng đặc tả TC-04: tranh 2 ghế còn lại 27
Bảng 15. Bảng đặc tả TC-05: cuộc hẹn cần xác nhận 27
Bảng 16. Bảng đặc tả TC-06: đổi lịch nhiều lần 27
Bảng 17. Bảng đặc tả TC-07: hủy và đổi lịch cùng lúc 28
Bảng 18. Bảng đặc tả TC-08: vượt giới hạn 1 booking mỗi ngày 28
Bảng 19. Bảng Booking 28
Bảng 20. Bảng Attendee 29
Bảng 21. Bảng BookingSeat 29
Bảng 22. Lịch thực hiện kế hoạch kiểm thử 29
Bảng 23. Cấu hình hai máy thực hiện kiểm thử 31
Bảng 24. Kết quả kiểm tra ba luồng nghiệp vụ (`npm run smoke`) 32
Bảng 25. Các lệnh chính của bộ test 33
Bảng 26. Kết quả chạy chính thức trên máy macOS 34
Bảng 27. Chi phí chạy kiểm thử đo được 38
Bảng 28. Ước lượng nhân lực (chưa phải số giờ thực tế) 38
Bảng 29. Danh sách lỗi và quan sát phụ 39
Bảng 30. Kết quả tái hiện độc lập từng lỗi 39

Bảng 31. Kết quả thí nghiệm thêm ràng buộc loại trừ 41
Bảng 32. Nhiệm vụ và đóng góp của từng thành viên 42
Bảng 33. Các thông tin dùng để chạy lại..... 43
Bảng 34. Bảng phân công công việc 47
Bảng 35. Tài liệu và bằng chứng đi kèm..... 47

MỞ ĐẦU

Trong thời đại số, việc đặt lịch hẹn trực tuyến đã trở thành nhu cầu phổ biến: khám bệnh, tư vấn, học thêm, họp công việc. Các hệ thống đặt lịch cho phép người sở hữu lịch công bố những khung giờ còn trống và để khách tự chọn giờ mà không cần trao đổi qua lại. Điều quan trọng nhất mà người dùng kỳ vọng ở loại hệ thống này rất đơn giản: một khung giờ chỉ được giữ cho một người.

Tuy nhiên, kỳ vọng đó chỉ dễ đạt được khi các yêu cầu đến lần lượt. Khi nhiều khách cùng bấm "Đặt lịch" trong một phần giây, hệ thống phải xử lý đồng thời nhiều yêu cầu cùng tranh một tài nguyên chung là lịch của chủ lịch. Nếu cách xử lý không an toàn trước tình huống này, hệ thống có thể nhận cả hai yêu cầu và chủ lịch bị đặt trùng giờ. Loại lỗi này rất khó gặp khi kiểm thử bằng tay hoặc bằng các kịch bản chạy tuần tự.

Xuất phát từ thực tế đó, nhóm thực hiện đề tài R05-K15 của môn Kiểm thử phần mềm: kiểm thử race condition cho Cal.com, một hệ thống đặt lịch mã nguồn mở. Nhóm dựng hệ thống trên máy cá nhân, phân tích luồng đặt lịch, thiết kế 8 test case xoay quanh các tính chất dữ liệu phải luôn đúng, viết bộ test tự động gửi request đồng thời bằng k6 và kiểm tra trực tiếp cơ sở dữ liệu PostgreSQL. Kết quả là 6 lỗi, mỗi lỗi đều được tái hiện lại bằng lệnh `curl` thuần và được xác nhận trên hai máy khác nhau.

Mục đích của báo cáo giữa kỳ này là **thuyết phục tính khả thi của kế hoạch kiểm thử** để được duyệt: kế hoạch có phạm vi hợp lý, tiêu chí đánh giá rõ ràng, chi phí thấp, và đã được chứng minh chạy thật bằng bộ test và các lỗi thật tìm được. Báo cáo cũng ghi rõ nhiệm vụ và đóng góp của từng thành viên.

CHƯƠNG 1. TỔNG QUAN VỀ ĐỀ TÀI

1.1. Giới thiệu đề tài

Cal.com là phần mềm đặt lịch hẹn mã nguồn mở, có chức năng tương tự dịch vụ Calendly. Chủ lịch (host) tạo các loại cuộc hẹn, chẳng hạn "Tư vấn 30 phút", và khai báo khung giờ mình rảnh. Hệ thống cung cấp một đường dẫn công khai cho từng loại cuộc hẹn. Khách mở đường dẫn, xem các giờ còn trống, chọn một giờ, điền họ tên và email rồi đặt lịch mà không cần tài khoản. Sau đó hệ thống ghi nhận cuộc hẹn, đánh dấu giờ đó đã kín và gửi thông báo cho hai bên. Khách cũng có thể đổi lịch hoặc hủy lịch bằng đường dẫn trong thông báo.

Đề tài R05-K15 chọn Cal.com làm hệ thống cần kiểm thử (mã R05 trong danh sách của giảng viên) và áp dụng kỹ thuật kiểm thử đồng thời, hay kiểm thử race condition (mã K15): tạo nhiều yêu cầu đồng thời để tìm các lỗi như đặt trùng giờ (double booking) hoặc ghi đè dữ liệu (lost update).

Phiên bản được kiểm thử là Cal.com `v6.2.0`, commit `1c193cca8682b33b9866c792186033f7ef886682`. Việc cố định phiên bản giúp kết quả kiểm thử không bị thay đổi khi mã nguồn trên nhánh chính được cập nhật trong học kỳ.

1.2. Lý do chọn đề tài

Thứ nhất, đặt lịch là bài toán điển hình của tranh chấp tài nguyên: nhiều người không quen biết nhau cùng giành một khung giờ của một chủ lịch. Hậu quả của lỗi đặt trùng giờ rất rõ ràng với người dùng: hai khách cùng nhận xác nhận cho một giờ, chủ lịch phải từ chối một người sau khi đã hứa.

Thứ hai, điểm vào đặt lịch của Cal.com là công khai, không cần đăng nhập, nên bất kỳ ai cũng gửi được nhiều yêu cầu cùng lúc, và hai khách thật hoàn toàn có thể đặt cùng một thời điểm. Theo đề bài của giảng viên, Cal.com đặc biệt phù hợp với kiểm thử trạng thái và kiểm thử đồng thời.

Thứ ba, các kịch bản kiểm thử chạy tuần tự thông thường không phát hiện được loại lỗi này, trong khi mã nguồn mở cho phép nhóm đọc trực tiếp luồng xử lý, xác định chỗ thiếu bảo vệ và giải thích nguyên nhân gốc của lỗi. Đây là cơ hội để

nhóm thực hành đầy đủ chu trình: phân tích, thiết kế kiểm thử, tự động hóa, phát hiện lỗi và phân tích nguyên nhân.

1.3. Mục tiêu đề tài

Mục tiêu chính của đề tài là áp dụng kỹ thuật kiểm thử đồng thời lên luồng đặt lịch của Cal.com, tìm ra các lỗi thật và chứng minh được chúng bằng dữ liệu kiểm chứng lại được. Cụ thể:

1) Dựng được Cal.com trên máy cá nhân, chạy thành công ba luồng nghiệp vụ (đặt lịch, đổi lịch, hủy lịch) và lập tài liệu cài đặt cho người khác làm lại.
2) Đọc hiểu mã nguồn, vẽ lại kiến trúc và luồng dữ liệu của nghiệp vụ trọng tâm, xác định các điểm có nguy cơ xảy ra tranh chấp.
3) Xác định các tính chất dữ liệu bất biến (invariant) và thiết kế danh sách test case để kiểm tra chúng.
4) Viết bộ test tự động, chạy lại được: gửi request đồng thời, kiểm tra dữ liệu trong cơ sở dữ liệu, có nhóm đối chứng gửi tuần tự.
5) Chạy test, thu thập kết quả, phát hiện lỗi, tái hiện lỗi độc lập và phân tích nguyên nhân gốc.
6) Làm việc nhóm theo quy trình chuẩn: dùng Git, bảng công việc, phân vai rõ ràng và báo cáo kỹ thuật.

1.4. Phạm vi nghiên cứu

Do hệ thống Cal.com rất lớn, đề tài giới hạn phạm vi vào luồng đặt lịch như trình bày ở Bảng 1, thay vì kiểm thử toàn bộ hệ thống.

Trong phạm vi	**Ngoài phạm vi**
Đặt lịch cuộc hẹn 1-1	Đồng bộ lịch ngoài (Google Calendar, Outlook)
Cuộc hẹn có ghế (nhiều người cùng một giờ, có giới hạn chỗ)	Thanh toán trực tuyến
Cuộc hẹn cần xác nhận của chủ lịch	Cuộc hẹn của nhóm, phân công vòng tròn (round-robin)
Đổi lịch và hủy lịch	API phiên bản 2 (NestJS)
Giới hạn số booking trong một ngày	Kiểm thử hiệu năng quy mô lớn

Bảng 1. Phạm vi kiểm thử của đề tài

Mọi lần chạy kiểm thử đều thực hiện trên máy cá nhân của nhóm, với một bản Cal.com và một cơ sở dữ liệu cục bộ. Nhóm không gửi tải vào bất kỳ máy chủ công khai nào.

1.5. Cấu trúc báo cáo và đối chiếu với yêu cầu của đề bài

Theo bản phân công đề tài, báo cáo phải có tám phần A đến H. Báo cáo này trình bày theo cấu trúc đồ án của khoa (mở đầu, bốn chương, kết luận) và đối chiếu các phần của đề bài với các mục tương ứng ở Bảng 2.

Phần	Yêu cầu của đề bài	Mục trong báo cáo
A	Mục tiêu hệ thống, tác nhân và use case chính, cây thư mục và module, phụ thuộc, luồng dữ liệu	3.2.1 đến 3.2.4
B	Ít nhất một sơ đồ kiến trúc mức component/container, giải thích 3 đến 5 module liên quan đến kiểm thử	3.2.5
C	Môi trường, phiên bản, lệnh chạy, dữ liệu mẫu; bằng chứng hệ thống chạy và ít nhất 3 luồng nghiệp vụ	4.2
D	Nguyên lý kỹ thuật, lý do chọn phạm vi, giả định, rủi ro, tiêu chí pass/fail	2.1, 1.2, 3.3.1 đến 3.3.3, 3.3.5
E	Thiết kế test case: input, tiền điều kiện, kết quả mong đợi, dữ liệu test	3.3.4 đến 3.3.7
F	Mã nguồn test chạy lại tự động, tệp README chứa lệnh chạy và phụ thuộc	4.1, 4.3
G	Log, báo cáo, số đo, danh sách lỗi và nguyên nhân gốc	4.4, 4.5
H	Ghim phiên bản, lưu cấu hình test, cách chạy lại trên máy mới	4.7

Bảng 2. Đối chiếu các phần A đến H của đề bài với các mục của báo cáo

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1. Race condition và kiểm thử đồng thời

Race condition (tranh chấp dữ liệu) xảy ra khi kết quả của chương trình phụ thuộc vào thứ tự xen kẽ của các thao tác đồng thời trên cùng một dữ liệu dùng chung. Dạng phổ biến nhất trong ứng dụng web là TOCTOU (time-of-check to time-of-use) [6]: yêu cầu đọc trạng thái, ra quyết định dựa trên trạng thái đó, rồi mới ghi. Nếu một yêu cầu khác ghi vào giữa hai bước đọc và ghi, quyết định ban đầu đã dựa trên dữ liệu cũ.

Ví dụ với hệ thống đặt lịch: yêu cầu A đọc lịch của chủ lịch và thấy khung 09:00 còn trống; yêu cầu B cũng đọc và thấy trống; sau đó cả hai cùng ghi booking. Kết quả là hai booking trùng giờ dù mỗi yêu cầu, xét riêng, đều "đúng".

Kiểm thử đồng thời khác kiểm thử chức năng thông thường ở ba điểm. Thứ nhất, kết quả đúng sai không thể chỉ nhìn vào mã phản hồi HTTP: một API có thể trả 200 cho mọi yêu cầu mà dữ liệu vẫn sai. Vì vậy người kiểm thử phải xác định các **invariant**, tức tính chất luôn phải đúng của dữ liệu (ví dụ "một chủ lịch không có hai booking còn hiệu lực chồng giờ") và kiểm tra trực tiếp trong cơ sở dữ liệu sau khi chạy. Thứ hai, các yêu cầu phải thực sự rơi vào cùng cửa sổ race, nên cần cơ chế đồng bộ thời điểm gửi. Thứ ba, kết quả mang tính xác suất, nên phải lặp nhiều vòng và báo tỉ lệ vòng vi phạm.

Để phân biệt lỗi race với lỗi logic thông thường, nhóm dùng **nhóm đối chứng**: gửi đúng các yêu cầu đó nhưng lần lượt. Nếu chạy tuần tự đúng mà chạy đồng thời sai thì đó là lỗi race; nếu cả hai đều sai thì đó là lỗi logic.

2.2. Hệ thống đặt lịch và các cơ chế bảo vệ

Một hệ thống đặt lịch có hai nhóm tác nhân: chủ lịch, người tạo loại cuộc hẹn và khai báo giờ rảnh; và khách, người chọn giờ và đặt lịch. Tài nguyên dùng chung là các khung giờ của chủ lịch. Để bảo vệ tài nguyên này khỏi tranh chấp, phần mềm thường kết hợp nhiều cơ chế ở các tầng khác nhau:

1) **Ràng buộc duy nhất (UNIQUE)** trong cơ sở dữ liệu: từ chối bản ghi thứ hai có cùng giá trị khóa. Hiệu quả khi hai yêu cầu trùng khít, nhưng không bắt được trường hợp chồng lấn một phần.

2) **Khóa dòng (`SELECT … FOR UPDATE`)** [4]: một giao dịch giữ khóa trên dòng dữ liệu, các giao dịch khác phải chờ. Dùng để tránh hai yêu cầu cùng đọc rồi cùng cập nhật một dòng.

3) **Ràng buộc loại trừ (exclusion constraint)** [3]: cho phép khai báo trong PostgreSQL rằng hai bản ghi không được có khoảng thời gian giao nhau với cùng một chủ lịch.

4) **Kiểm tra ở tầng ứng dụng**: đọc dữ liệu hiện có và từ chối nếu vi phạm quy tắc. Đây là cơ chế yếu nhất trước tranh chấp nếu không đi kèm khóa hoặc ràng buộc ở cơ sở dữ liệu.

2.3. Công nghệ sử dụng

2.3.1. Cal.com

Cal.com là monorepo quản lý bằng Yarn 4 và Turborepo [1]. Ứng dụng web viết bằng TypeScript, chạy trên Next.js 16; lớp dữ liệu dùng Prisma 6.16.1 kết nối PostgreSQL; giao tiếp giữa giao diện và máy chủ dùng thêm tRPC. Điểm vào đặt lịch công khai là `POST /api/book/event`, điểm vào hủy lịch là `POST /api/cancel` (có cơ chế CSRF kiểu double-submit cookie). Phần xử lý nghiệp vụ nằm trong gói `packages/features/bookings`.

2.3.2. PostgreSQL

PostgreSQL 16 là hệ quản trị cơ sở dữ liệu quan hệ lưu toàn bộ dữ liệu của Cal.com và cũng là nơi nhóm kiểm tra các invariant. PostgreSQL hỗ trợ ràng buộc duy nhất, khóa dòng và ràng buộc loại trừ [3][4]. Cơ sở dữ liệu được cấu hình tối đa 300 kết nối đồng thời, vì mỗi yêu cầu gửi đồng thời giữ một kết nối.

2.3.3. k6

k6 là công cụ kiểm thử tải mã nguồn mở của Grafana Labs, viết kịch bản bằng JavaScript [2]. Nhóm dùng bộ thực thi `per-vu-iterations`: mỗi người dùng ảo

(VU) gửi đúng một yêu cầu, và tất cả cùng chờ một mốc thời gian chung rồi mới gửi để các yêu cầu thực sự chồng lên nhau. Phiên bản dùng là k6 2.3.0 trên máy macOS và 2.2.0 trên máy Windows.

2.3.4. Node.js và thư viện pg

Phần điều phối của bộ test viết bằng Node.js 22, gồm chuẩn bị dữ liệu, gọi k6, truy vấn cơ sở dữ liệu bằng thư viện `pg` để kiểm invariant và sinh báo cáo. Node.js 22.23.2 trên máy macOS và 22.18.0 trên máy Windows.

2.3.5. Git và GitHub Projects

Mã nguồn bộ test, tài liệu và kết quả được quản lý bằng Git và lưu trên GitHub. Công việc của nhóm được theo dõi bằng bảng công việc GitHub Projects. Chương 4 trình bày kết quả sử dụng.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ KIỂM THỬ

3.1. Phân tích yêu cầu kiểm thử

Quá trình phân tích chia thành hai nhóm: các nghiệp vụ của hệ thống cần được kiểm thử (yêu cầu chức năng) và các yêu cầu đối với chính bộ kiểm thử (yêu cầu phi chức năng).

3.1.1. Yêu cầu chức năng

Các nghiệp vụ trong phạm vi kiểm thử gồm:

1) Đặt lịch cuộc hẹn 1-1: khách chọn một giờ trống và đặt; hệ thống phải bảo đảm một giờ chỉ thuộc về một khách.
2) Đặt chỗ cuộc hẹn có ghế: nhiều người có thể cùng đặt một giờ cho đến khi hết số ghế quy định.
3) Đặt lịch cuộc hẹn cần xác nhận: booking ở trạng thái chờ xác nhận vẫn giữ chỗ cho đến khi chủ lịch xử lý.
4) Đổi lịch: chuyển một booking sang giờ mới, booking cũ chuyển sang trạng thái đã hủy.
5) Hủy lịch: khách hủy booking bằng đường dẫn hủy.
6) Giới hạn số booking: loại cuộc hẹn có thể quy định tối đa một số booking trong một ngày.

3.1.2. Yêu cầu phi chức năng của bộ test

1) Đồng thời thật: các yêu cầu phải được gửi trong cùng một khoảng thời gian rất ngắn để chạm vào cửa sổ race.
2) Oracle là dữ liệu: kết luận đúng sai dựa trên kiểm tra trực tiếp cơ sở dữ liệu, không dựa vào mã HTTP.
3) Chạy lại được: một lệnh chạy toàn bộ bộ test, có tài liệu cài đặt từ máy sạch và cố định phiên bản.
4) Cô lập: mỗi vòng dùng một ngày riêng trong tương lai để các vòng không ảnh hưởng nhau.

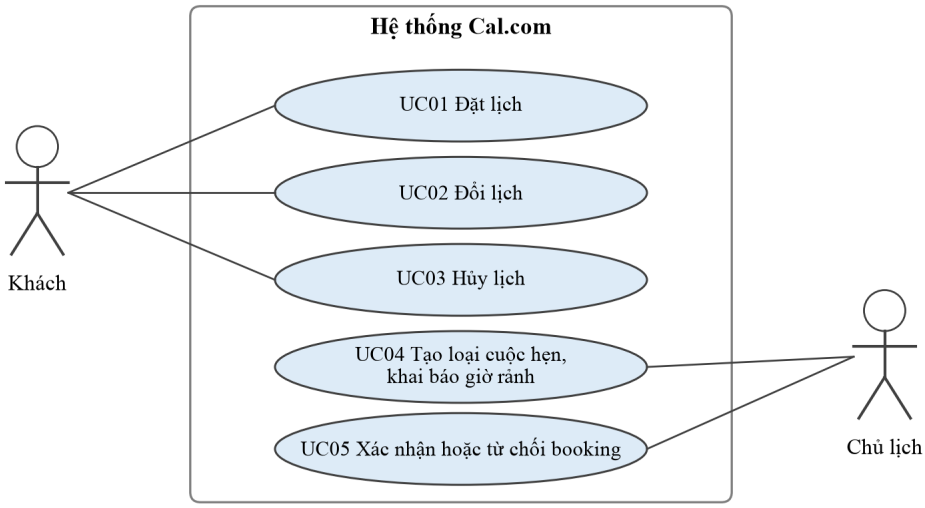
5) An toàn: chỉ chạy trên máy cục bộ, không tạo tải vào máy chủ công khai.

6) Có đối chứng: mỗi test case chạy cả chế độ đồng thời và tuần tự để phân loại lỗi.

3.2. Phân tích hệ thống được kiểm thử

3.2.1. Mục tiêu, tác nhân và use case chính

Mục tiêu của Cal.com là cho phép người sở hữu lịch công bố giờ rảnh và để khách tự đặt, đổi, hủy cuộc hẹn mà không cần trao đổi qua lại. Hệ thống có hai tác nhân chính: **khách** (không cần tài khoản) và **chủ lịch** (có tài khoản). Hình 1 và Bảng 3 trình bày các use case chính; đề tài tập trung vào nhóm use case của khách (UC01 đến UC03) vì đó là nơi nhiều người cùng tranh một tài nguyên.



Hình 1. Sơ đồ use case chính của Cal.com

Mã	Use case	Tác nhân	Mô tả
UC01	Đặt lịch	Khách	Chọn giờ trống của một loại cuộc hẹn, điền họ tên, email và đặt
UC02	Đổi lịch	Khách	Chuyển cuộc hẹn đã đặt sang giờ khác bằng đường dẫn đổi lịch
UC03	Hủy lịch	Khách	Hủy cuộc hẹn đã đặt bằng đường dẫn hủy
UC04	Tạo loại cuộc hẹn, khai báo giờ rảnh	Chủ lịch	Cấu hình thời lượng, số ghế, giới hạn, giờ làm việc
UC05	Xác nhận hoặc từ chối booking	Chủ lịch	Xử lý các booking ở trạng thái chờ xác nhận

Bảng 3. Các use case chính của Cal.com

3.2.2. Cây thư mục và module

Cal.com là monorepo với khoảng 7 950 tệp TypeScript. Bảng 4 liệt kê các thư mục chính liên quan đến luồng đặt lịch.

Thư mục	Vai trò
`apps/web`	Ứng dụng Next.js: giao diện và các API route (`/api/book/event`, `/api/cancel`)
`apps/api/v1`, `apps/api/v2`	REST API phiên bản 1 và phiên bản 2 (NestJS)
`packages/features`	Nghiệp vụ theo miền: bookings, availability, slots, schedules, eventtypes…
`packages/trpc`	Các router tRPC cho giao diện (ví dụ lấy giờ trống, giữ chỗ tạm)
`packages/prisma`	Lược đồ cơ sở dữ liệu, migration và các phần mở rộng (extension)
`packages/lib`	Tiện ích dùng chung (lỗi HTTP, kiểm soát tần suất…)
`packages/app-store`	Các tích hợp: lịch ngoài, video, thanh toán
`packages/emails`, `packages/ui`	Thông báo và các thành phần giao diện

Bảng 4. Các thư mục chính của Cal.com

3.2.3. Phụ thuộc chính

Thành phần	Phiên bản	Vai trò
Node.js	22	Môi trường chạy
Yarn	4.12.0	Quản lý gói, khai báo trong `packageManager`
Next.js	16.1.5	Khung ứng dụng web
React	18.2.0	Thư viện giao diện
Prisma	6.16.1	ORM, kết nối PostgreSQL
PostgreSQL	16	Cơ sở dữ liệu
tRPC, NestJS, Turborepo	–	Giao tiếp giao diện – máy chủ, API v2, điều phối build

Bảng 5. Các phụ thuộc chính của Cal.com v6.2.0

3.2.4. Luồng dữ liệu của nghiệp vụ đặt lịch

1) Khách mở trang của loại cuộc hẹn; giao diện gọi tRPC `slots.getSchedule` để tính các giờ trống từ giờ làm việc trừ đi các booking hiện có.

2) Khách chọn giờ; giao diện có thể gọi `slots.reserveSlot` để giữ chỗ tạm trên giao diện.

3) Khách điền thông tin và gửi `POST /api/book/event`, vào `RegularBookingService.createBooking()`: (a) tải loại cuộc hẹn, chủ lịch và booking gốc nếu đổi lịch; (b) kiểm tra giới hạn số booking; (c) kiểm tra chủ lịch còn trống; (d) nếu cuộc hẹn có ghế thì thêm ghế vào booking của giờ đó; (e) ngược lại mở giao dịch, hủy booking gốc nếu đổi lịch rồi ghi `Booking` và `Attendee`.

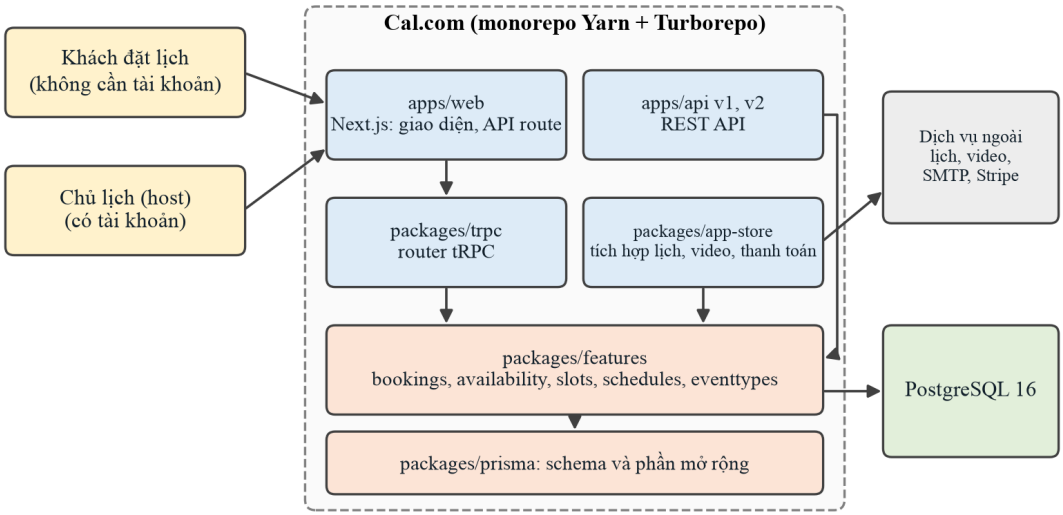
4) Sau khi ghi, hệ thống gửi email, kích hoạt webhook và đồng bộ lịch ngoài.

5) Hủy lịch: `POST /api/cancel` (kèm mã CSRF) chuyển trạng thái booking sang đã hủy và xóa khóa chống trùng.

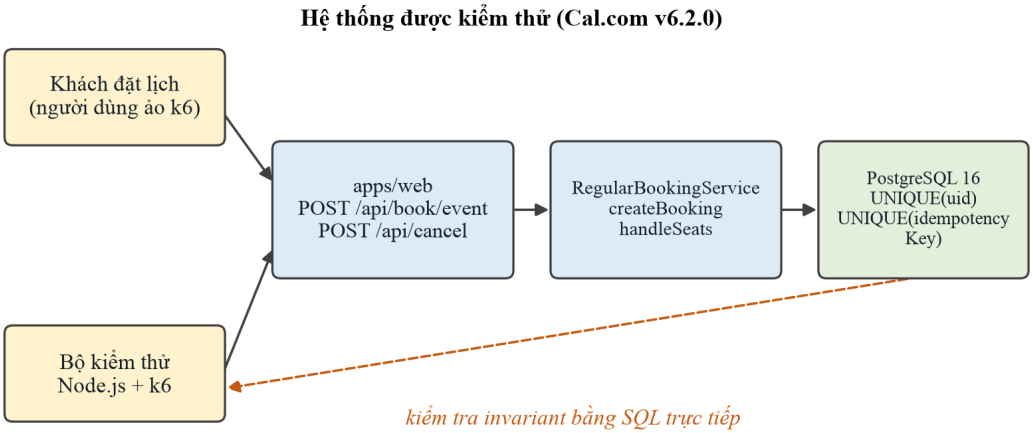
Các bước (b), (c) chỉ **đọc**, bước (e) mới **ghi**, và giữa chúng không có khóa nào được giữ. Đây là nơi nhóm kỳ vọng tìm thấy race.

3.2.5. Kiến trúc và các module liên quan đến kiểm thử

Hình 2 là sơ đồ mức container của Cal.com, và Hình 3 mô tả vị trí của bộ test: bộ test đóng vai khách gửi yêu cầu vào `apps/web`, đồng thời đọc trực tiếp PostgreSQL để kiểm invariant.



Hình 2. Sơ đồ container của Cal.com



Hình 3. Hệ thống được kiểm thử và vị trí của bộ test

Năm module liên quan trực tiếp đến kiểm thử:

1) `RegularBookingService.createBooking()`: điều phối toàn bộ luồng tạo và đổi lịch. Nó đọc giới hạn số booking, đọc lịch trống rồi mới ghi.

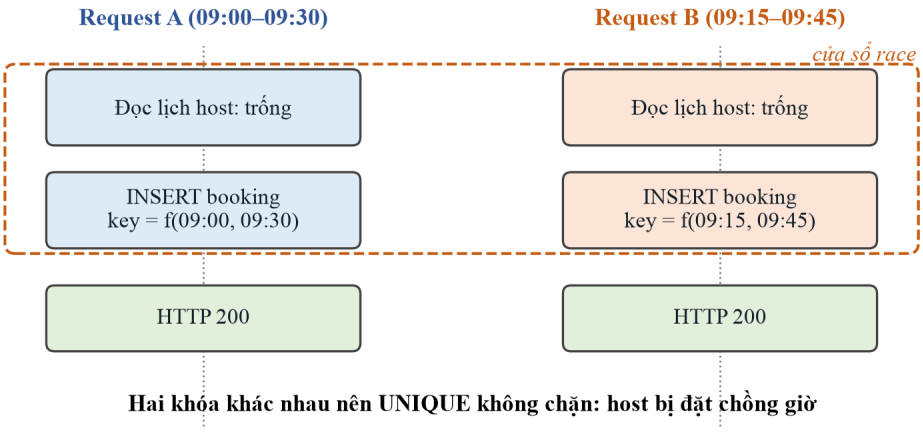
2) `createBooking.ts`: ghi `Booking` trong một giao dịch; khi đổi lịch, đồng thời chuyển booking gốc sang đã hủy bằng lệnh cập nhật theo mã (`id`), không kèm điều kiện trạng thái.

3) Phần mở rộng Prisma `booking-idempotency-key`: gán `idempotencyKey` là giá trị băm từ giờ bắt đầu, giờ kết thúc và chủ lịch, chỉ khi booking ở trạng thái đã chấp nhận (ACCEPTED). Cột này có ràng buộc duy nhất.

4) `handleSeats` và `addSeatToBooking`: xử lý cuộc hẹn có ghế; hàm thêm ghế dùng `SELECT … FOR UPDATE` khi booking của giờ đó đã tồn tại.

5) `checkBookingLimits`, `ensureAvailableUsers`, `getOriginalRescheduledBooking`: các bước kiểm tra trước khi ghi (giới hạn, lịch trống, booking gốc hợp lệ).

Khi đọc mã, nhóm nhận thấy giữa các bước kiểm tra (đọc) và bước ghi không có khóa hay giao dịch bao trùm. Hình 4 minh họa tình huống hai khách đặt hai giờ chồng nhau: cả hai cùng thấy lịch trống, mỗi bên ghi một booking với khóa `idempotencyKey` khác nhau, nên ràng buộc duy nhất không chặn được. Đây là giả thuyết ban đầu của nhóm, được kiểm chứng ở Chương 4.



Hình 4. Khe hở kiểm tra rồi mới ghi khi hai khách đặt hai giờ chồng nhau

3.3. Thiết kế kiểm thử

3.3.1. Nguyên lý và lý do chọn phạm vi

Nguyên lý của kỹ thuật được trình bày ở mục 2.1: gửi đồng thời thật, dùng invariant trong cơ sở dữ liệu làm oracle, có nhóm đối chứng tuần tự và lặp nhiều vòng. Nhóm chọn phạm vi luồng đặt lịch vì ba lý do ở mục 1.2: tranh chấp tài nguyên là bản chất của bài toán, điểm vào công khai nên dễ bị khai thác đồng thời, và có thể đọc mã để giải thích nguyên nhân. Phạm vi gồm năm biến thể có cơ chế bảo vệ khác nhau (cuộc hẹn 1-1, có ghế, cần xác nhận, đổi hoặc hủy lịch, giới hạn số lượng), giúp so sánh chỗ nào đã được bảo vệ và chỗ nào chưa.

3.3.2. Giả định

1) Một bản Cal.com, một PostgreSQL, chạy trên máy cục bộ; không có bộ nhớ đệm hay hàng đợi phân tán.
2) Chạy bằng `yarn dev`. Logic nghiệp vụ giống bản dựng chính thức; độ trễ có thể khác. Trên hệ thống thật, cửa sổ race có thể rộng hơn do có thêm lời gọi tới lịch ngoài.
3) Chủ lịch test không kết nối lịch ngoài nên lịch bận chỉ đến từ bảng `Booking`.
4) Kiểm soát tần suất yêu cầu (rate limit) tắt, vì không đặt biến `UNKEY_ROOT_KEY`.
5) Mỗi vòng dùng một ngày riêng trong tương lai (2027 đến 2029) nên các vòng độc lập nhau.

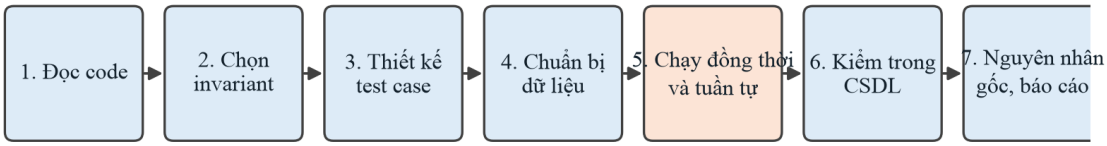
3.3.3. Rủi ro và cách xử lý

Rủi ro	**Cách xử lý**
Race không tái hiện ổn định	Chạy 10 vòng mỗi test case, báo tỉ lệ vòng vi phạm
Nhầm lỗi logic với lỗi race	Nhóm đối chứng gửi tuần tự cùng các yêu cầu
Request không thực sự đồng thời	Mốc thời gian gửi chung `START_AT`, đo độ lệch thời điểm gửi (0 đến 1 ms)
Oracle sai (ví dụ lệch múi giờ)	Kiểm invariant bằng SQL; phát hiện và sửa lỗi lệch 7 giờ của trình điều khiển
Kết quả khác trên máy khác	Ghim phiên bản, chạy lại trên máy thứ hai
Hai lượt test chạy chồng nhau làm hỏng kết quả	Quy tắc: tại một thời điểm chỉ một lượt chạy; kiểm tra tiến trình trước khi chạy
Máy yếu khi chạy cả hệ thống	Giảm số yêu cầu đồng thời; khởi động nóng Cal.com

Bảng 6. Rủi ro của kế hoạch kiểm thử và cách xử lý

3.3.4. Quy trình test

Quy trình gồm bảy bước như Hình 5. Điểm khác biệt so với quy trình kiểm thử chức năng nằm ở bước 5 và 6: chạy cả hai chế độ đồng thời và tuần tự, rồi kiểm tra dữ liệu trong cơ sở dữ liệu.



Bước 5 chạy cả hai chế độ: đồng thời (k6) và tuần tự (nhóm đối chứng)

Hình 5. Quy trình test gồm bảy bước

Bước	Việc làm	Sản phẩm
1	Đọc mã luồng đặt lịch, tìm chỗ kiểm tra rồi mới ghi	Sơ đồ kiến trúc, giả thuyết
2	Chọn 7 invariant I1 đến I7	Bảng 8
3	Thiết kế 8 test case TC-01 đến TC-08	Mục 3.3.6
4	Tạo host test, loại cuộc hẹn; mỗi vòng dùng ngày riêng	`fixtures.sql`
5	Chạy đồng thời bằng k6, rồi chạy lại đúng các yêu cầu đó tuần tự	`run.mjs`, `race.js`
6	Truy vấn SQL kiểm invariant sau mỗi vòng	`scenarios.mjs`, `db.mjs`
7	Đọc mã tìm nguyên nhân, thử khắc phục trên dữ liệu test	Chương 4

Bảng 7. Các bước của quy trình test

3.3.5. Invariant và tiêu chí pass/fail

ID	Tính chất luôn phải đúng	Cách kiểm trong CSDL
I1	Một chủ lịch không có hai booking hiệu lực chồng giờ	Ghép bảng `Booking` với chính nó theo `userId`, hai khoảng giờ giao nhau
I2	Không vượt số ghế của cuộc hẹn	Đếm `BookingSeat` theo giờ
I2b	Còn ghế thì người đặt không bị từ chối	So số người thành công với số ghế trống
I3	Một giờ có tối đa một booking chờ xác nhận	Như I1, gồm cả trạng thái `pending`
I4	Một booking gốc sinh tối đa một booking mới hiệu lực	Đếm booking có `fromReschedule` bằng mã gốc
I5	Hủy báo thành công thì không còn booking hiệu lực của cuộc hẹn đó	Kết hợp mã HTTP lệnh hủy và trạng thái các booking con
I6	Số booking trong ngày không vượt giới hạn	Đếm booking hiệu lực theo ngày

ID	Tính chất luôn phải đúng	Cách kiểm trong CSDL
I7	Không có phản hồi 5xx, quá thời gian hay lỗi kết nối	Log của k6

Bảng 8. Các invariant dùng làm tiêu chí đúng sai

Tiêu chí pass/fail:

1) Một vòng đạt khi mọi invariant của test case đúng và không có phản hồi 5xx hay lỗi kết nối.

2) Một test case đạt khi cả 10 vòng đạt. Chỉ cần một vòng vi phạm là test case không đạt.

3) Là lỗi race khi chạy đồng thời không đạt nhưng chạy tuần tự đạt. Nếu chạy tuần tự cũng không đạt thì là lỗi logic.

3.3.6. Danh sách và đặc tả test case

Dữ liệu chung của mọi test case: chủ lịch `k15host` (múi giờ UTC, rảnh cả ngày mọi ngày) và năm loại cuộc hẹn ở Bảng 9. Mỗi yêu cầu dùng một email người đặt khác nhau.

Loại cuộc hẹn	Thời lượng	Cấu hình đặc biệt
`k15-30`	30 phút	Không
`k15-60`	60 phút	Không
`k15-seated`	30 phút	3 ghế mỗi giờ
`k15-confirm`	30 phút	Cần xác nhận, booking chờ vẫn giữ chỗ
`k15-limit`	30 phút	Tối đa 1 booking mỗi ngày

Bảng 9. Các loại cuộc hẹn dùng cho kiểm thử

Bảng 10 tổng hợp danh sách test case. Từ Bảng 11 đến Bảng 18 là đặc tả từng test case theo mẫu: mô tả, tác nhân, tiền điều kiện, input, kết quả mong đợi, invariant.

TC	Test case	Input (gửi đồng thời)	Kết quả mong đợi	Invariant
TC-01	Nhiều người đặt cùng một giờ	10 người đặt `k15-30` lúc 09:00	1 người 200, 9 người 409	I1
TC-02	Các giờ chồng lấn	10 người, giờ lệch 3 phút, độ dài 30 và 60 phút	Chỉ 1 booking hiệu lực	I1
TC-03	Tranh ghế đầu tiên	6 người đặt `k15-seated`, giờ trống	3 người 200, 3 người 409; 3 ghế	I2, I2b

TC	Test case	Input (gửi đồng thời)	Kết quả mong đợi	Invariant
TC-04	Tranh 2 ghế còn lại	6 người, giờ đã có 1 người	2 người 200, 4 người 409	I2, I2b
TC-05	Cuộc hẹn cần xác nhận	10 người đặt `k15-confirm`	Chỉ 1 booking chờ xác nhận	I3
TC-06	Đổi lịch nhiều lần	5 lệnh đổi lịch cùng một booking	1 lệnh thành công	I4
TC-07	Hủy và đổi lịch cùng lúc	1 lệnh hủy + 1 lệnh đổi lịch	Không thể cả hai cùng thành công	I5
TC-08	Vượt giới hạn 1 booking mỗi ngày	8 người đặt 8 giờ khác nhau	Chỉ 1 người thành công	I6

Bảng 10. Danh sách test case

Mô tả	Kiểm tra ràng buộc duy nhất của hệ thống khi nhiều yêu cầu trùng khít cùng giờ.
Tác nhân chính	Khách đặt lịch (người dùng ảo k6)
Tiền điều kiện	Giờ 09:00 của chủ lịch còn trống.
Input	10 người cùng đặt `k15-30` lúc 09:00.
Kết quả mong đợi	Đúng 1 yêu cầu thành công (200); 9 yêu cầu bị từ chối (409); cơ sở dữ liệu có 1 booking.
Invariant	I1, I7

Bảng 11. Bảng đặc tả TC-01: nhiều người đặt cùng một giờ

Mô tả	Kiểm tra hệ thống có chặn các booking chồng giờ nhưng không trùng khít (lệch giờ bắt đầu, khác độ dài).
Tác nhân chính	Khách đặt lịch (người dùng ảo k6)
Tiền điều kiện	Khung 08:00 đến 11:00 còn trống.
Input	10 người: 7 người đặt `k15-30` bắt đầu 09:00 + 3i phút, 3 người đặt `k15-60` bắt đầu 08:30 + 3i phút; mọi khoảng chứa mốc 09:29.
Kết quả mong đợi	Chỉ 1 booking hiệu lực trong khung; các yêu cầu còn lại bị từ chối.
Invariant	I1, I7

Bảng 12. Bảng đặc tả TC-02: các giờ chồng lấn

Mô tả	Kiểm tra cuộc hẹn có ghế khi nhiều người cùng đặt vào giờ chưa có ai.
Tác nhân chính	Khách đặt lịch (người dùng ảo k6)
Tiền điều kiện	Giờ trống, cuộc hẹn có 3 ghế.
Input	6 người cùng đặt `k15-seated`.

Kết quả mong đợi	3 người thành công, 3 người bị từ chối vì hết ghế; một booking duy nhất với 3 ghế.
Invariant	I2, I2b, I7

Bảng 13. Bảng đặc tả TC-03: tranh ghế đầu tiên

Mô tả	Kiểm tra cơ chế khóa dòng khi giờ đã có booking và còn 2 ghế.
Tác nhân chính	Khách đặt lịch (người dùng ảo k6)
Tiền điều kiện	Giờ đã có 1 người đặt (đặt tuần tự trước).
Input	6 người cùng đặt `k15-seated`.
Kết quả mong đợi	Đúng 2 người thành công, 4 người bị từ chối; tổng 3 ghế.
Invariant	I2, I2b, I7

Bảng 14. Bảng đặc tả TC-04: tranh 2 ghế còn lại

Mô tả	Kiểm tra booking chờ xác nhận có giữ chỗ khi nhiều người đặt cùng giờ.
Tác nhân chính	Khách đặt lịch (người dùng ảo k6)
Tiền điều kiện	Giờ trống, cuộc hẹn cần xác nhận và bật giữ chỗ cho booking chờ.
Input	10 người cùng đặt `k15-confirm`.
Kết quả mong đợi	Chỉ 1 booking chờ xác nhận cho giờ đó.
Invariant	I3, I7

Bảng 15. Bảng đặc tả TC-05: cuộc hẹn cần xác nhận

Mô tả	Kiểm tra một booking đã đổi lịch có thể bị đổi tiếp bằng cùng mã gốc hay không.
Tác nhân chính	Khách đặt lịch (người dùng ảo k6)
Tiền điều kiện	Booking B lúc 09:00 ở trạng thái đã chấp nhận.
Input	5 lệnh đổi lịch B sang 10:00, 11:00, 12:00, 13:00, 14:00.
Kết quả mong đợi	Một lệnh thành công; B bị hủy; chỉ 1 booking mới hiệu lực.
Invariant	I4, I7

Bảng 16. Bảng đặc tả TC-06: đổi lịch nhiều lần

Mô tả	Kiểm tra tính nhất quán khi hai thao tác trái ngược tác động lên cùng một booking.
Tác nhân chính	Khách đặt lịch (người dùng ảo k6)
Tiền điều kiện	Booking B lúc 09:00.
Input	1 lệnh hủy B và 1 lệnh đổi lịch B sang 11:00 gửi đồng thời.

Kết quả mong đợi	Không thể cả hai cùng thành công; nếu hủy thành công thì không còn booking hiệu lực.
Invariant	I5, I7

Bảng 17. Bảng đặc tả TC-07: hủy và đổi lịch cùng lúc

Mô tả	Kiểm tra giới hạn số booking khi nhiều yêu cầu cùng đếm trước khi ghi.
Tác nhân chính	Khách đặt lịch (người dùng ảo k6)
Tiền điều kiện	Ngày trống, cuộc hẹn giới hạn 1 booking mỗi ngày.
Input	8 người cùng đặt 8 giờ khác nhau (09:00 đến 16:00) trong cùng một ngày.
Kết quả mong đợi	Chỉ 1 người thành công, 7 người bị từ chối vì vượt giới hạn.
Invariant	I6, I7

Bảng 18. Bảng đặc tả TC-08: vượt giới hạn 1 booking mỗi ngày

3.3.7. Cơ sở dữ liệu liên quan

Các bảng của Cal.com mà bộ test đọc để kiểm invariant gồm `Booking`, `Attendee` và `BookingSeat`; Bảng 19 đến Bảng 21 liệt kê các cột được sử dụng. Hai ràng buộc duy nhất đáng chú ý nằm ở bảng `Booking`: `uid` và `idempotencyKey`.

Trường	Kiểu dữ liệu	Mô tả
id	integer	Khóa chính
uid	text	Mã công khai của booking (duy nhất)
userId	integer	Chủ lịch sở hữu booking
eventTypeId	integer	Loại cuộc hẹn
startTime, endTime	timestamp	Giờ bắt đầu và kết thúc, lưu theo UTC
status	enum	Trạng thái: accepted, pending, cancelled, rejected
fromReschedule	text	Mã booking gốc nếu được tạo do đổi lịch
rescheduled	boolean	Đánh dấu booking đã được đổi lịch
idempotencyKey	text	Khóa chống trùng (duy nhất), chỉ gán khi booking đã chấp nhận

Bảng 19. Bảng Booking

Trường	Kiểu dữ liệu	Mô tả
id	integer	Khóa chính
bookingId	integer	Booking mà người tham dự thuộc về
email, name	text	Email và họ tên người đặt
timeZone	text	Múi giờ của người tham dự

Bảng 20. Bảng Attendee

Trường	Kiểu dữ liệu	Mô tả
id	integer	Khóa chính
referenceUid	text	Mã tham chiếu của ghế
bookingId	integer	Booking chứa ghế
attendeeId	integer	Người tham dự giữ ghế

Bảng 21. Bảng BookingSeat

3.4. Kế hoạch kiểm thử và tính khả thi

3.4.1. Kế hoạch và lịch thực hiện

Giai đoạn	Việc	Trạng thái
Đăng ký	Chốt thành viên, repo, kỹ thuật, phiên bản; lập bảng công việc	Xong
Sprint 1 (đến 20/10)	Dựng hệ thống, vẽ kiến trúc, viết tài liệu cài đặt, kế hoạch test, thử nghiệm đầu tiên	Xong
Sprint 2 và 3	Bộ test 8 test case, chạy 10 vòng, đối chứng tuần tự, đo cửa sổ race, chạy lại trên máy Windows	Xong
Sprint 2 và 3 (còn lại)	Kiểm thử qua giao diện bằng Playwright	Chưa làm
Sprint 4	Rà soát nguyên nhân gốc, đề xuất sửa, hoàn thiện báo cáo cuối	Đang làm
Tháng 12	Slide bảo vệ, tập dượt, đánh giá chéo giữa các thành viên	Chưa làm

Bảng 22. Lịch thực hiện kế hoạch kiểm thử

3.4.2. Tính khả thi và đề nghị duyệt

Nhóm đề nghị được duyệt kế hoạch kiểm thử trên cơ sở các bằng chứng sau, đều đã thực hiện thật:

1) **Hệ thống dựng được và chạy được:** Cal.com v6.2.0 chạy trên hai máy (macOS và Windows 11), ba luồng đặt, đổi, hủy lịch đều thành công.
2) **Kỹ thuật phát hiện được lỗi thật:** 6 trên 8 test case phát hiện lỗi; cả 6 lỗi tái hiện độc lập bằng `curl`, có dữ liệu sai trong cơ sở dữ liệu.
3) **Kết quả không phụ thuộc máy:** máy Windows cho cùng kết luận 8/8 với máy macOS.
4) **Chi phí thấp:** 0 đồng, lượt chạy chính thức dưới 3 phút máy, bộ test khoảng 970 dòng mã.
5) **Rủi ro đã được kiểm soát:** nhóm đối chứng tuần tự, lặp 10 vòng, kiểm dữ liệu trong cơ sở dữ liệu, ghim phiên bản.
6) **Phạm vi và việc còn lại rõ ràng:** phần chưa làm (Playwright, báo lỗi cho Cal.com) đã được liệt kê cùng lịch thực hiện.

CHƯƠNG 4. XÂY DỰNG VÀ THỰC HIỆN KIỂM THỬ

4.1. Cấu trúc thư mục và tổ chức mã nguồn

Mã nguồn bộ test nằm trong thư mục `race-tests` của kho `kiem_thu_pm`. Mã của Cal.com được tải riêng theo tag `v6.2.0` và không đưa vào kho. Hình 6 mô tả cấu trúc thư mục.

```
race-tests/
├── fixtures/fixtures.sql        tạo host k15host và 5 loại cuộc hẹn
├── k6/race.js                   gửi request đồng thời (mỗi VU 1 request)
├── src/
│   ├── config.mjs               địa chỉ Cal.com, chuỗi kết nối CSDL
│   ├── api.mjs                  dựng request đặt, đổi, hủy lịch
│   ├── db.mjs                   truy vấn SQL kiểm invariant
│   ├── scenarios.mjs            8 test case: chuẩn bị và kiểm tra
│   ├── run.mjs                  điều phối: chuẩn bị, k6, kiểm CSDL, ghi kết quả
│   ├── report.mjs               sinh REPORT.md
│   ├── window.mjs               đo độ rộng cửa sổ race
│   ├── smoke.mjs                kiểm tra 3 luồng nghiệp vụ
│   └── demo-midterm.mjs         demo giữa kỳ
├── repro/repro-curl.sh          tái hiện độc lập bằng curl
├── experiments/                 thí nghiệm khắc phục
└── results/                     kết quả các lần chạy
```

Hình 6. Cấu trúc thư mục bộ test

4.2. Môi trường, triển khai và bằng chứng hệ thống chạy

Thành phần	Máy 1 (macOS)	Máy 2 (Windows)
Máy	MacBook Apple Silicon, 10 nhân, 32 GB RAM	MSI, Intel Core i7-13620H 2,40 GHz, 16 GB RAM (15,7 GB dùng được)
Hệ điều hành	macOS	Windows 11, 64-bit
Cal.com	v6.2.0 (`1c193cca`)	v6.2.0 (`1c193cca`)
Node.js	22.23.2	22.18.0
k6	2.3.0	2.2.0 (windows/amd64)
PostgreSQL	16 (cổng 5432, 300 kết nối)	Cài đặt riêng
Người chạy	Nhóm	Ngô Quốc Trường Vĩ

Bảng 23. Cấu hình hai máy thực hiện kiểm thử

Các bước dựng hệ thống từ máy sạch được ghi trong tệp `README.md` của kho: tải mã Cal.com đúng tag, tạo cơ sở dữ liệu `calendso`, cấu hình biến môi trường (`DATABASE_URL`, `NEXTAUTH_SECRET`, `CALENDSO_ENCRYPTION_KEY`), cài thư

viện, chạy `yarn workspace @calcom/prisma db-deploy` và `db-seed`, rồi chạy `yarn dev`. Nhóm gặp và xử lý bốn khó khăn:

1) Máy macOS không có Docker, nên nhóm dùng Postgres.app và khởi tạo cụm cơ sở dữ liệu riêng.

2) Các cột thời gian của Cal.com là `timestamp without time zone` lưu giờ UTC. Trình điều khiển `pg` mặc định đổi theo múi giờ máy (+07:00) làm các truy vấn kiểm invariant bị lệch 7 giờ. Nhóm ép UTC trong `db.mjs`.

3) API hủy lịch yêu cầu mã CSRF dài 64 ký tự theo kiểu double-submit cookie; bộ test gửi cùng một mã ở cả cookie và nội dung yêu cầu.

4) Ở chế độ phát triển, Cal.com biên dịch route lần đầu được gọi nên yêu cầu đầu tiên rất chậm. Bộ test thêm bước khởi động nóng: đặt rồi hủy một booking thử trước khi chạy.

Để chứng minh ba luồng nghiệp vụ chạy được, nhóm viết lệnh `npm run smoke` chạy lần lượt đặt lịch, đổi lịch, hủy lịch rồi đặt lại giờ vừa hủy. Kết quả ở Bảng 24: cả bốn bước đều trả HTTP 200 và trạng thái trong cơ sở dữ liệu đúng.

Luồng	HTTP	Thời gian	Trạng thái trong cơ sở dữ liệu
1. Đặt lịch	200	104 ms	accepted
2. Đổi lịch	200	92 ms	booking gốc cancelled, booking mới accepted
3. Hủy lịch	200	65 ms	cancelled
4. Đặt lại giờ vừa hủy	200	69 ms	giờ được giải phóng, đặt được

Bảng 24. Kết quả kiểm tra ba luồng nghiệp vụ (`npm run smoke`)

4.3. Xây dựng bộ test

4.3.1. Dữ liệu test

Tệp `fixtures.sql` tạo lại chủ lịch `k15host` và năm loại cuộc hẹn ở Bảng 9; có thể chạy nhiều lần mà không để lại dữ liệu cũ. Lệnh `npm run fixtures` nạp tệp này. Cuộc hẹn `k15-confirm` bật cờ giữ chỗ cho booking chờ xác nhận để có một invariant rõ ràng cho TC-05.

4.3.2. Gửi yêu cầu đồng thời bằng k6

Tập yêu cầu của mỗi vòng do `run.mjs` sinh ra rồi truyền cho `race.js`. Mỗi người dùng ảo lấy đúng một yêu cầu theo số thứ tự của mình, chờ tới mốc thời gian `START_AT` chung, gửi rồi ghi kết quả. Độ lệch thời điểm gửi giữa các yêu cầu của một vòng đo được từ 0 đến 1 mili-giây.

```
// rào chắn thời gian: mọi VU chờ tới cùng một mốc rồi mới gửi
const waitMs = startAt + (req.delayMs || 0) - Date.now();
if (waitMs > 0) sleep(waitMs / 1000);
const res = http.request(req.method, req.url,
JSON.stringify(req.body), { headers: req.headers });
```

4.3.3. Kiểm tra invariant và nhóm đối chứng

Sau mỗi vòng, `scenarios.mjs` truy vấn trực tiếp PostgreSQL để kiểm invariant. Ví dụ I1 được kiểm bằng cách ghép bảng `Booking` với chính nó để tìm các cặp booking hiệu lực của cùng một chủ lịch có khoảng giờ giao nhau:

```
SELECT a.uid, b.uid FROM "Booking" a JOIN "Booking" b
  ON a."userId" = b."userId" AND a.id < b.id
 AND a."startTime" < b."endTime" AND b."startTime" < a."endTime"
WHERE a.status IN ('accepted','pending') AND b.status IN
('accepted','pending');
```

Chế độ `--mode sequential` gửi đúng tập yêu cầu đó lần lượt thay vì đồng thời; kết quả chạy tuần tự là nhóm đối chứng để phân loại lỗi.

4.3.4. Báo cáo và lệnh chạy

Lệnh	**Tác dụng**
`npm run fixtures`	Tạo lại chủ lịch và loại cuộc hẹn test
`npm run smoke`	Kiểm tra ba luồng nghiệp vụ
`npm run race`	Chạy 8 test case đồng thời, 10 vòng mỗi test case
`npm run race -- --mode sequential`	Chạy đối chứng tuần tự
`node src/window.mjs`	Đo độ rộng cửa sổ race
`npm run demo:midterm`	Demo giữa kỳ: kiểm hệ thống, ba luồng và thử nghiệm race (khoảng 20 giây)
`bash repro/repro-curl.sh`	Tái hiện độc lập D2, D3, D5, D6 bằng `curl`

Bảng 25. Các lệnh chính của bộ test

Mỗi lần chạy sinh thư mục kết quả gồm `REPORT.md` (bảng tổng hợp và chi tiết từng vòng), `summary.json` và thư mục `raw/` chứa yêu cầu đã gửi, phản hồi

từng yêu cầu và tóm tắt của k6. Mã thoát bằng 0 khi mọi test case đạt, 1 khi có test case không đạt, 2 khi bộ test lỗi.

4.4. Thực hiện kiểm thử và kết quả

4.4.1. Kết quả chạy đồng thời và tuần tự

Lượt chạy chính thức ngày 18/10/2026 trên máy macOS: 8 test case, mỗi test case 10 vòng ở mỗi chế độ. Bảng 26 tổng hợp kết quả; Hình 7 so sánh với máy Windows.

TC	Đồng thời (vòng vi phạm)	Tuần tự (vòng vi phạm)	Kết luận	Lỗi
TC-01	0/10	0/10	Đúng	–
TC-02	10/10	0/10	Lỗi race	D1
TC-03	10/10	0/10	Lỗi race	D2
TC-04	0/10	0/10	Đúng	–
TC-05	10/10	0/10	Lỗi race	D3
TC-06	10/10	10/10	Lỗi logic	D4
TC-07	10/10	0/10	Lỗi race	D5
TC-08	10/10	0/10	Lỗi race	D6

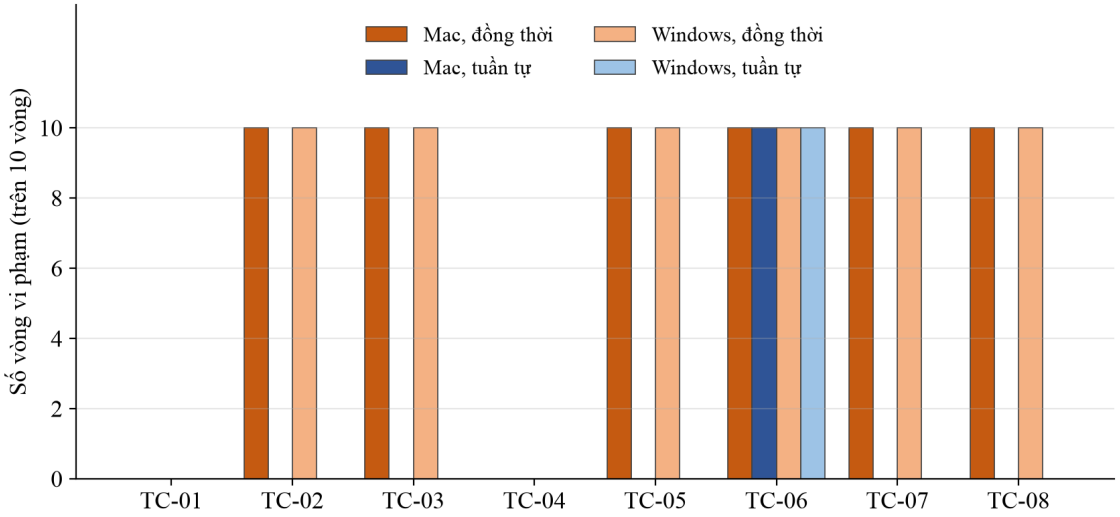
Bảng 26. Kết quả chạy chính thức trên máy macOS

Nhận xét:

1) TC-01 và TC-04 đạt ở cả hai chế độ: ràng buộc duy nhất ở TC-01 và khóa dòng ở TC-04 hoạt động đúng. Kết quả này cũng cho thấy bộ test không báo lỗi giả.
2) Năm test case TC-02, 03, 05, 07, 08 chỉ không đạt khi chạy đồng thời: đây là lỗi race.
3) TC-06 không đạt ở cả hai chế độ nên là lỗi logic, không phải race.
4) Ở TC-02, mã HTTP của cả 10 yêu cầu đều là 200 nhưng cơ sở dữ liệu có 10 booking chồng giờ, tạo ra 45 cặp chồng nhau trong mỗi vòng. Đây là lý do nhóm kiểm tra dữ liệu thay vì tin mã HTTP.

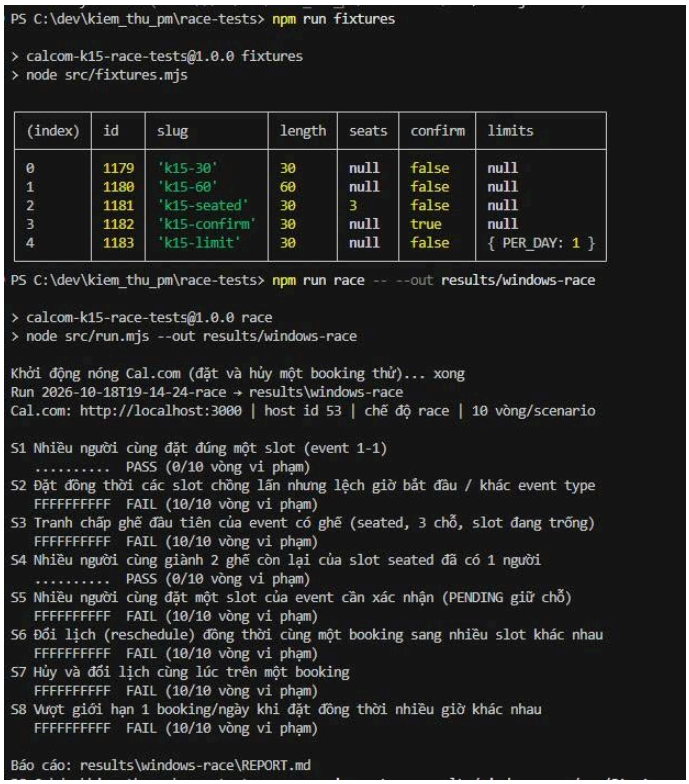
4.4.2. Chạy lại trên máy Windows

Thành viên Ngô Quốc Trường Vĩ chạy lại toàn bộ trên máy Windows 11 (Bảng 23) với cùng phiên bản mã. Kết quả khớp 8/8 với máy macOS: số vòng vi phạm và dữ liệu trong cơ sở dữ liệu ở vòng 1 giống nhau, trừ TC-05, nơi máy macOS nhận 9 booking chờ xác nhận và máy Windows nhận 10; cả hai đều vi phạm 10/10 vòng.

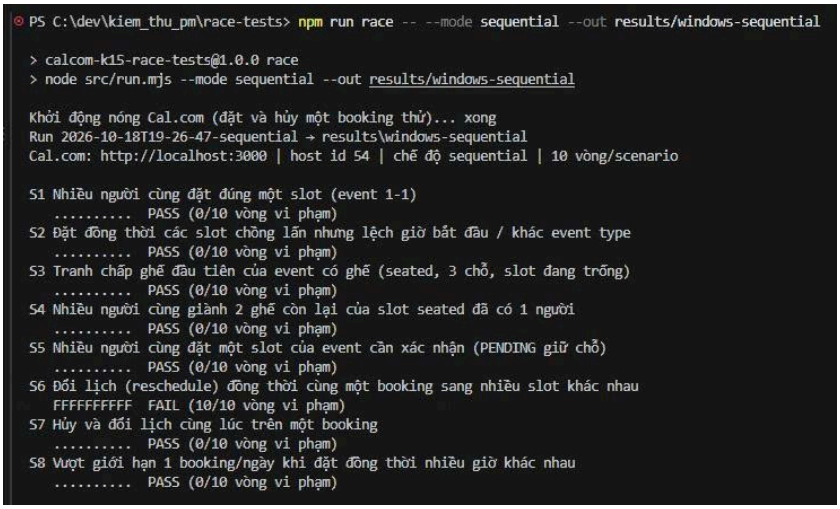


Hình 7. Số vòng vi phạm của từng test case trên hai máy, hai chế độ

Máy Windows chạy chậm hơn: lượt đồng thời mất 210 giây so với 154 giây trên máy macOS, lượt tuần tự 43 giây so với 23 giây. Độ trễ p95 của TC-01 là 9 703 ms trên Windows so với 417 ms trên macOS; nhiều khả năng do Cal.com biên dịch lần đầu ở vòng đầu, nhóm chưa kiểm chứng. Hình 8 và Hình 9 là ảnh chụp Terminal của hai lượt chạy trên máy Windows.



Hình 8. Kết quả chạy đồng thời trên máy Windows



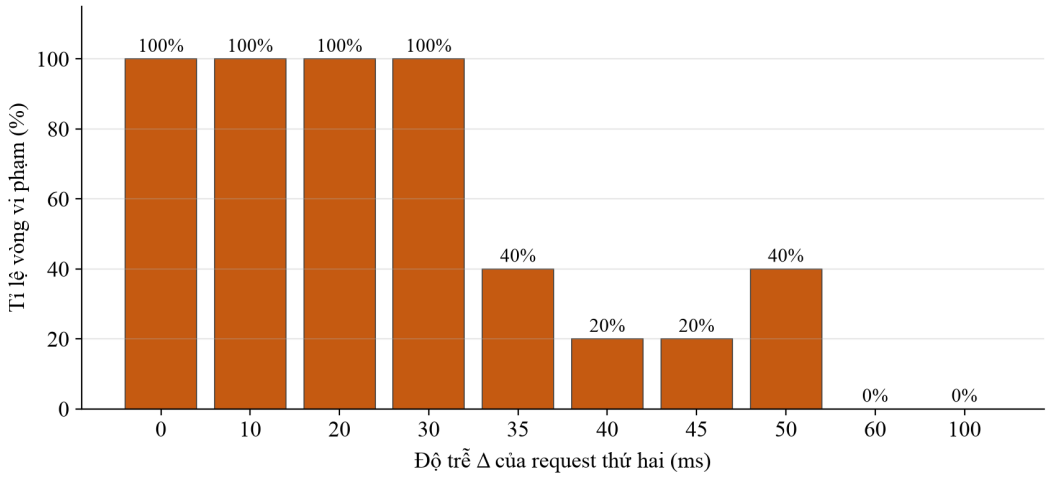
Hình 9. Kết quả chạy tuần tự trên máy Windows

Hai lượt chạy đầu trên máy Windows không hợp lệ và đã bị loại: lượt thứ nhất do máy chưa cài k6, lượt thứ hai do Cal.com dừng giữa chừng và test case TC-01 báo không đạt bất thường. Kết quả ở trên là lượt chạy sạch sau khi khắc phục. Từ đó bộ test được bổ sung bước khởi động nóng và thông báo rõ ràng khi Cal.com dừng giữa lúc chạy.

4.4.3. Độ nhạy và cửa sổ race

Để xem lỗi có cần tải lớn hay không, nhóm giảm xuống chỉ 2 yêu cầu đồng thời và chạy 20 vòng. TC-02 (các giờ chồng lấn) vi phạm ở 20/20 vòng và TC-08 (giới hạn mỗi ngày) vi phạm ở 20/20 vòng. Như vậy hai khách đặt gần cùng lúc là đủ gây lỗi.

Để đo độ rộng cửa sổ race, nhóm cho hai yêu cầu xung đột, yêu cầu thứ hai gửi trễ Δ mili-giây. Hình 10 cho thấy với Δ từ 0 đến 30 ms vòng nào cũng vi phạm; từ 35 đến 50 ms vi phạm một phần; từ 60 ms trở đi không còn. Vậy cửa sổ race khoảng 30 đến 60 ms, xấp xỉ thời gian xử lý một yêu cầu trên máy test. Trên hệ thống thật, thời gian này dài hơn vì có thêm lời gọi tới lịch ngoài, nên cửa sổ có thể rộng hơn.



Hình 10. Tỉ lệ vòng vi phạm theo độ trễ của yêu cầu thứ hai

4.4.4. Chi phí kiểm thử

Chi phí tiền là 0 đồng, vì toàn bộ công cụ miễn phí hoặc mã nguồn mở và chạy trên máy cá nhân. Chi phí chạy test được đo từ thời điểm bắt đầu và kết thúc ghi trong `summary.json` của từng lần chạy (Bảng 27).

Lần chạy	Vòng	Request	Thời gian máy
Đồng thời chính (macOS)	80	570	154 giây
Tuần tự đối chứng (macOS)	80	570	23 giây
Độ nhạy: 2 request, 20 vòng	100	200	164 giây
Thí nghiệm khắc phục	40	380	85 giây
Đo cửa sổ race (18 mốc độ trễ)	180	360	300 giây
Tổng		1 710	khoảng 12 phút

Bảng 27. Chi phí chạy kiểm thử đo được

Chạy lại bộ test chính (cả đồng thời và tuần tự) tốn dưới 3 phút. Chi phí xây dựng bộ test khoảng 970 dòng mã (Node.js, k6, SQL). Về nhân lực, nhóm chưa theo dõi giờ làm của từng người; Bảng 28 là ước lượng khối lượng công việc, không phải số giờ đo thực tế.

Hạng mục	**Người phụ trách**	**Ước lượng (giờ)**
Dựng Cal.com, seed data, chạy 3 luồng nghiệp vụ	Nguyễn Thanh Tâm	6 đến 8
Đọc mã, vẽ kiến trúc, phân tích nguyên nhân gốc	Võ Công Vinh	6 đến 8
Thiết kế test case, viết bộ test	Nguyễn Đức Tín	10 đến 14
Chạy test, ghi kết quả, báo cáo, slide	Ngô Quốc Trường Vĩ	12 đến 16
Tổng		34 đến 46 (khoảng 9 đến 12 giờ mỗi người)

Bảng 28. Ước lượng nhân lực (chưa phải số giờ thực tế)

4.5. Phân tích lỗi

4.5.1. Danh sách lỗi

Lỗi	**Hiện tượng**	**Loại**	**Mức độ**
D1	Chủ lịch bị đặt chồng giờ khi các giờ lệch nhau hoặc khác loại cuộc hẹn	Race	Cao
D2	Cuộc hẹn có ghế: còn ghế nhưng người đặt sau bị từ chối với thông báo xung đột	Race	Trung bình
D3	Nhiều booking chờ xác nhận cho cùng một giờ	Race	Trung bình
D4	Một booking gốc đã đổi lịch vẫn đổi lịch tiếp được, sinh nhiều booking mới	Logic	Cao
D5	Hủy và đổi lịch cùng lúc đều báo thành công, vẫn còn booking hiệu lực	Race	Trung bình
D6	Vượt giới hạn 1 booking mỗi ngày khi đặt đồng thời	Race	Trung bình
O1	Vượt giới hạn booking trả mã 401 (chưa xác thực) thay vì 403	Mã lỗi	Thấp
O2	Mã booking `uid` sinh từ tên chủ lịch, giờ và thời điểm hiện tại, có thể trùng nếu hai yêu cầu xử lý cùng một mili-giây	Thiết kế	Thấp

Bảng 29. Danh sách lỗi và quan sát phụ

4.5.2. Tái hiện độc lập bằng curl

Để chắc rằng lỗi không do bộ test, nhóm tái hiện từng lỗi bằng lệnh `curl` thuần trên Cal.com đang chạy, rồi đối chiếu dữ liệu trong cơ sở dữ liệu (Bảng 30). Ví dụ với D1, hai khách đặt hai giờ chồng nhau cùng lúc:

```
curl -X POST localhost:3000/api/book/event -d '{... khách A,
09:00–09:30 ...}' &
curl -X POST localhost:3000/api/book/event -d '{... khách B,
09:15–09:45 ...}' &
wait
```

Cả hai yêu cầu đều trả HTTP 200 và cơ sở dữ liệu có hai booking đã chấp nhận chồng giờ của cùng một chủ lịch (09:00 đến 09:30 và 09:15 đến 09:45). Khi gửi tuần tự hai yêu cầu chồng giờ tương tự, yêu cầu thứ hai bị từ chối với HTTP 409. Hệ thống có kiểm tra chồng giờ, nhưng chỉ đúng khi các yêu cầu đến lần lượt.

Lỗi	Tái hiện bằng curl	Dữ liệu trong CSDL
D1	2 khách đặt 2 giờ chồng nhau cùng lúc: cả hai 200	2 booking đã chấp nhận chồng giờ
D2	6 người đặt cuộc hẹn 3 ghế: 1 trả 200, 5 trả 409	Chỉ bán 1 trên 3 ghế
D3	10 người đặt cuộc hẹn cần xác nhận: 8 trả 200	8 booking chờ xác nhận cùng một giờ
D4	Đổi lịch liên tiếp một booking gốc 3 lần: cả 3 trả 200	3 booking đã chấp nhận từ cùng một gốc (cả khi gửi tuần tự)
D5	Hủy và đổi lịch cùng lúc: cả hai 200	Booking 09:00 đã hủy nhưng vẫn còn booking 11:00 hiệu lực
D6	8 người đặt 8 giờ, giới hạn 1 mỗi ngày: cả 8 trả 200	8 booking hiệu lực trong một ngày

Bảng 30. Kết quả tái hiện độc lập từng lỗi

4.5.3. Nguyên nhân gốc

Nhóm lỗi race D1, D3, D5, D6 có chung nguyên nhân là mẫu kiểm tra rồi mới ghi không có khóa. Trong `RegularBookingService`, các bước đọc lịch trống (`ensureAvailableUsers`), đọc giới hạn số booking và đọc trạng thái booking gốc

đều thực hiện trước, rồi `createBooking` mới ghi, giữa hai bước không có giao dịch hay khóa bao trùm.

Lớp bảo vệ duy nhất trong cơ sở dữ liệu là ràng buộc duy nhất trên `idempotencyKey`. Khóa này tính từ giờ bắt đầu, giờ kết thúc và chủ lịch nên chỉ bắt được hai yêu cầu trùng khít (TC-01). Hai khoảng 09:00–09:30 và 09:15–09:45 cho ra hai khóa khác nhau nên không bị chặn (D1). Thêm vào đó, phần mở rộng chỉ gán khóa cho booking ở trạng thái đã chấp nhận; booking chờ xác nhận có khóa rỗng, và cơ sở dữ liệu cho phép nhiều giá trị rỗng trong cột duy nhất, nên D3 không có lớp bảo vệ nào ở tầng dữ liệu. Với D6, phép đếm số booking trong ngày và lệnh ghi là hai bước tách rời.

D2 (cuộc hẹn có ghế): `handleSeats` tìm booking của giờ đó; nếu đã có thì gọi `addSeatToBooking` có khóa dòng (TC-04 đạt). Nếu chưa có, mọi yêu cầu cùng thấy "chưa có booking" và cùng cố tạo mới; yêu cầu đầu thắng, các yêu cầu sau trùng khóa và bị ánh xạ thành lỗi 409 mà không thử lại theo nhánh thêm ghế.

D4 là lỗi logic: hàm `getOriginalRescheduledBooking` chỉ từ chối khi booking gốc đã hủy mà chưa được đánh dấu đổi lịch. Trạng thái "đã hủy và đã đánh dấu đổi lịch" vừa dùng cho trường hợp chủ lịch yêu cầu khách đổi giờ (cần cho phép đổi), vừa là trạng thái của booking đã được đổi xong (cần chặn). Hàm không phân biệt hai trường hợp nên booking đã đổi vẫn đổi tiếp được. Câu lệnh cập nhật booking gốc trong giao dịch cũng chỉ theo `id`, không kèm điều kiện trạng thái.

O1: ở `checkBookingLimits.ts` lỗi vượt giới hạn (mã 403) bị bọc lại thành 401. **O2:** mã booking `uid` được sinh từ tên chủ lịch, giờ bắt đầu và thời điểm hiện tại; hai yêu cầu cùng giờ trong cùng một mili-giây sinh cùng `uid` và một yêu cầu bị từ chối ngẫu nhiên. Nhóm đếm được 45 lần lỗi trùng `uid` trong nhật ký máy chủ trong quá trình chạy test.

4.5.4. Thí nghiệm khắc phục

Để kiểm chứng nguyên nhân, nhóm thêm vào cơ sở dữ liệu test một ràng buộc loại trừ không cho hai booking hiệu lực (accepted hoặc pending) của cùng chủ lịch có khoảng giờ giao nhau [3], rồi chạy lại các test case. Kết quả ở Bảng 31.

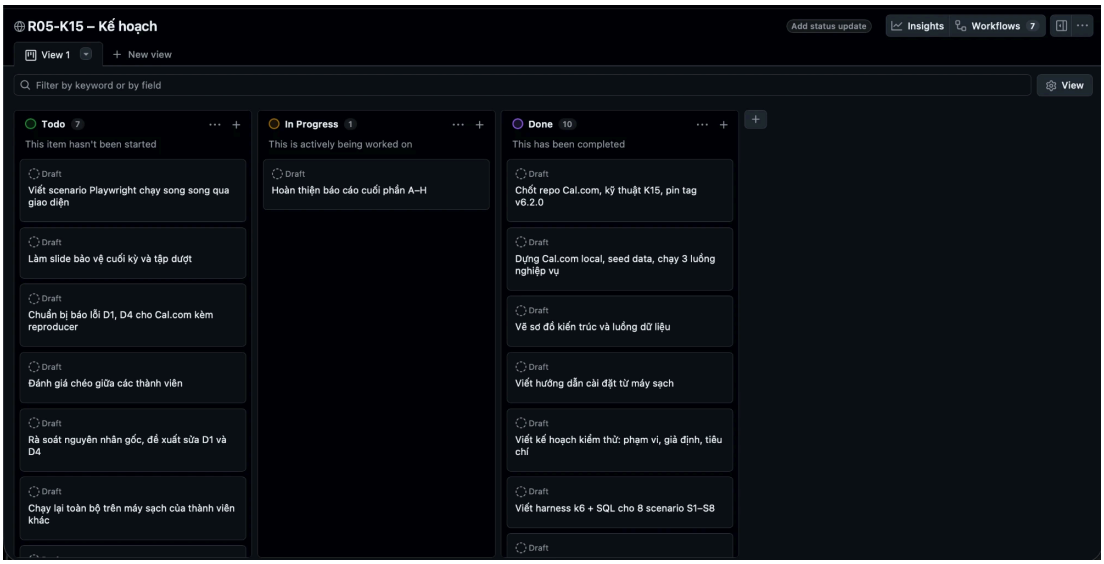
Test case	Trước	Sau khi thêm ràng buộc	Nhận xét
TC-02 (D1)	10 booking chồng	1 booking	9 yêu cầu thua nhận HTTP 500
TC-05 (D3)	7 đến 10 booking chờ	1 booking chờ	9 yêu cầu thua nhận HTTP 500
TC-08 (D6)	8 booking mỗi ngày	8 booking mỗi ngày	Không đổi, cần khóa theo chủ lịch và ngày

Bảng 31. Kết quả thí nghiệm thêm ràng buộc loại trừ

Dữ liệu trở nên toàn vẹn ở D1 và D3, xác nhận nguyên nhân là thiếu ràng buộc ở cơ sở dữ liệu. Tuy nhiên người thua nhận lỗi 500 thay vì 409, vì mã Cal.com chỉ ánh xạ lỗi vi phạm ràng buộc duy nhất sang 409, chưa xử lý mã lỗi của ràng buộc loại trừ. Vì vậy cần sửa ở cả hai tầng: ràng buộc ở cơ sở dữ liệu và ánh xạ lỗi ở ứng dụng. D6 là bài toán đếm nên cần cơ chế khác như khóa theo chủ lịch và ngày (advisory lock) hoặc giao dịch mức serializable có thử lại. Đây là hướng đề xuất; nhóm chưa sửa mã Cal.com.

4.6. Quản lý dự án, nhiệm vụ và đóng góp của từng thành viên

Nhóm làm việc theo mô hình Scrum rút gọn, họp ngắn đầu và cuối tuần. Mã nguồn bộ test và tài liệu được quản lý trên GitHub (kho `2312741-sudo/kiem_thu_pm`). Công việc được theo dõi trên bảng GitHub Projects "R05-K15 – Kế hoạch" với ba cột Todo, In Progress, Done và 18 thẻ (Hình 11). Các thẻ hiện chưa gán người phụ trách và lịch sử commit chủ yếu thuộc một thành viên; nhóm sẽ bổ sung trước buổi bảo vệ.



Hình 11. Bảng công việc GitHub Projects của nhóm

Bảng 32 ghi nhiệm vụ và đóng góp của từng thành viên theo phân vai của nhóm.

Họ tên	Vai trò	Nhiệm vụ đã thực hiện	Sản phẩm
Nguyễn Thanh Tâm 2312741	Môi trường và DevOps	Dựng Cal.com, cơ sở dữ liệu, seed data; ghim phiên bản; chạy test trên máy macOS	README, tài liệu cài đặt, kết quả chạy chính thức
Võ Công Vinh 2312800	Phân tích hệ thống	Đọc mã, vẽ kiến trúc và luồng dữ liệu; phân tích nguyên nhân gốc; đề xuất sửa	Sơ đồ, `DEFECTS.md`, thí nghiệm khắc phục
Nguyễn Đức Tín 2312774	Tác giả test	Thiết kế invariant và test case; viết bộ test (k6, Node.js, SQL)	`race-tests/`, đặc tả test case
Ngô Quốc Trường Vĩ 2312796	Chạy test và báo cáo	Chạy test, ghi kết quả; chạy lại trên máy Windows; viết báo cáo, slide, hướng dẫn	`windows-race`, báo cáo, slide

Bảng 32. Nhiệm vụ và đóng góp của từng thành viên

4.7. Khả năng tái lập

Để người khác chạy lại được trên máy mới, nhóm ghim và lưu các thông tin sau:

Hạng mục	Cách cố định
Phiên bản Cal.com	Tag `v6.2.0`, commit `1c193cca8682b33b9866c792186033f7ef886682`
Môi trường	Node.js 22, Yarn 4.12.0 (khai báo `packageManager`), PostgreSQL 16, k6 2.x

Hạng mục	Cách cố định
Phụ thuộc của bộ test	`race-tests/package.json` và `package-lock.json` (thư viện `pg` 8.x)
Dữ liệu test	`fixtures.sql` tạo lại từ đầu; mỗi lần chạy xóa booking của chủ lịch test; mỗi vòng dùng ngày riêng
Cấu hình test	Tham số dòng lệnh `--scenarios --rounds --n --stagger --mode --out`, ghi lại trong `summary.json`
Kết quả	`summary.json`, `REPORT.md`, yêu cầu đã gửi và log k6 thô của từng vòng

Bảng 33. Các thông tin dùng để chạy lại

Chạy lại trên máy mới gồm các bước: dựng Cal.com theo `README.md` (khoảng 15 phút, phần lớn là `yarn install`); cài k6; chạy `npm install`, `npm run fixtures`, `npm run race` và `npm run race -- --mode sequential` trong thư mục `race-tests`; so sánh kết quả với Bảng 26. Hướng dẫn riêng cho máy Windows nằm ở `docs/HUONG_DAN_CHAY_LAI_WINDOWS.md`. Vì race mang tính xác suất, tỉ lệ có thể lệch đôi chút giữa các máy; với các lỗi đã báo cáo, cửa sổ race (30 đến 60 ms) rộng hơn nhiều so với độ lệch thời điểm gửi (nhỏ hơn 1 ms) nên kết luận ổn định.

KẾT LUẬN

1. Kết quả đạt được

Nhóm đã dựng thành công Cal.com v6.2.0 trên hai máy (macOS và Windows 11), chạy được ba luồng nghiệp vụ đặt lịch, đổi lịch, hủy lịch và lập tài liệu cài đặt từ máy sạch. Nhóm đã phân tích kiến trúc và luồng xử lý của nghiệp vụ trọng tâm, xác định bảy tính chất dữ liệu bất biến và thiết kế 8 test case với đặc tả đầy đủ tiền điều kiện, dữ liệu, kết quả mong đợi.

Bộ test tự động gồm khoảng 970 dòng mã, gửi yêu cầu đồng thời bằng k6, kiểm tra trực tiếp cơ sở dữ liệu và có nhóm đối chứng gửi tuần tự. Kết quả lượt chạy chính thức: 6 trên 8 test case phát hiện lỗi, gồm 5 lỗi race (D1, D2, D3, D5, D6) và 1 lỗi logic (D4), cùng hai quan sát phụ. Cả 6 lỗi đều được tái hiện độc lập bằng `curl` thuần với dữ liệu sai nhìn thấy được trong cơ sở dữ liệu, và kết quả trên máy Windows khớp 8/8 với máy macOS.

Nhóm cũng xác định được nguyên nhân gốc: mẫu kiểm tra rồi mới ghi không có khóa, ràng buộc chống trùng quá hẹp và một trạng thái mang hai nghĩa. Thí nghiệm thêm ràng buộc loại trừ trên dữ liệu test chứng minh nguyên nhân cho D1 và D3, đồng thời chỉ ra hệ thống còn thiếu xử lý mã lỗi tương ứng.

Với các bằng chứng trên, nhóm đề nghị được duyệt kế hoạch kiểm thử đã trình bày ở mục 3.4: hệ thống chạy được, kỹ thuật phát hiện được lỗi thật, kết quả không phụ thuộc máy, chi phí thấp và phần việc còn lại đã rõ ràng.

2. Hạn chế và hướng phát triển

Đề tài còn một số hạn chế. Thứ nhất, nhóm chưa báo các lỗi tìm được cho nhóm phát triển Cal.com, chưa kiểm tra trên phiên bản mới hơn và chưa sửa mã Cal.com, nên chưa thể khẳng định phía Cal.com xếp tất cả là lỗi (ví dụ D2 có thể được xem là hành vi chấp nhận được). Thứ hai, bộ test chạy ở chế độ phát triển (`yarn dev`) trên một bản Cal.com và một cơ sở dữ liệu cục bộ, chưa thử trên bản dựng chính thức hay nhiều máy chủ sau bộ cân bằng tải. Thứ ba, nhóm chưa kiểm thử qua giao diện bằng Playwright, chưa kiểm thử API phiên bản 2, luồng nhóm và điểm giữ chỗ tạm. Thứ tư, nhân lực chỉ là ước lượng vì nhóm chưa theo dõi giờ làm.

Hướng phát triển gồm: bổ sung kịch bản Playwright chạy song song qua giao diện để minh họa lỗi D1 trên giao diện thật; rà soát và đề xuất bản sửa cho D1 và D4 (ràng buộc loại trừ, ánh xạ lỗi, kiểm tra lại trạng thái trong giao dịch khi đổi và hủy lịch, khóa theo chủ lịch cho giới hạn số booking); báo lỗi cho nhóm phát triển Cal.com kèm bộ tái hiện; và đưa bộ test vào quy trình tích hợp liên tục nhờ mã thoát của lệnh chạy.

TÀI LIỆU THAM KHẢO

[1] Cal.com. (n.d.). cal.com [Mã nguồn]. GitHub. https://github.com/calcom/cal.com

[2] Grafana Labs. (n.d.). Grafana k6 documentation. https://grafana.com/docs/k6/latest/

[3] The PostgreSQL Global Development Group. (n.d.). PostgreSQL 16 Documentation: Constraints. https://www.postgresql.org/docs/16/ddl-constraints.html

[4] The PostgreSQL Global Development Group. (n.d.). PostgreSQL 16 Documentation: Explicit Locking. https://www.postgresql.org/docs/16/explicit-locking.html

[5] Prisma. (n.d.). Transactions and batch queries. https://www.prisma.io/docs/orm/prisma-client/queries/transactions

[6] MITRE. (n.d.). CWE-367: Time-of-check Time-of-use (TOCTOU) Race Condition. https://cwe.mitre.org/data/definitions/367.html

[7] Nguyễn Thế Lâm. (2026). Hướng dẫn thực hiện đề tài môn Kiểm thử phần mềm [Tài liệu môn học]. Trường Đại học Đà Lạt.

PHỤ LỤC

Phân công công việc

Họ tên	Công việc
Nguyễn Thanh Tâm	- Dựng Cal.com, cơ sở dữ liệu, seed data, tài liệu cài đặt - Ghim phiên bản, môi trường chạy test trên máy macOS
Võ Công Vinh	- Đọc mã, vẽ kiến trúc và luồng dữ liệu - Phân tích nguyên nhân gốc, đề xuất sửa lỗi
Nguyễn Đức Tín	- Thiết kế invariant và test case - Viết bộ test (k6, Node.js, SQL)
Ngô Quốc Trường Vĩ	- Chạy test, ghi kết quả, chạy lại trên máy Windows - Viết báo cáo, slide, tài liệu hướng dẫn

Bảng 34. Bảng phân công công việc

Tài liệu và bằng chứng đi kèm (trong kho https://github.com/2312741-sudo/kiem_thu_pm)

Nội dung	Đường dẫn
Hướng dẫn cài đặt và chạy lại	`README.md`, `docs/HUONG_DAN_CHAY_LAI_WINDOWS.md`
Quy trình test đầy đủ	`docs/QUY_TRINH_TEST.md`
Hướng dẫn người test, phiếu ghi kết quả	`docs/HUONG_DAN_NGUOI_TEST.md`
Kết quả test giữa kỳ (sinh tự động)	`docs/KET_QUA_TEST_GIUA_KY.md`
Mô tả chi tiết từng lỗi	`docs/DEFECTS.md`, `docs/MINH_CHUNG_LOI.md`
Kết quả thô lượt chạy chính thức	`race-tests/results/giua-ky-race/`, `giua-ky-sequential/`
Kết quả máy Windows	`race-tests/results/windows-race/`, `windows-sequential/`
Tái hiện độc lập bằng curl	`race-tests/repro/repro-curl.sh`, `docs/evidence/repro-curl.txt`
Bảng công việc	https://github.com/users/2312741-sudo/projects/1

Bảng 35. Tài liệu và bằng chứng đi kèm